```
================================================================================
 oil_network — design principles & next steps
 Offline reading notes for flight work, snapshot 2026-05-13
================================================================================
```

PURPOSE OF THIS DOCUMENT

A condensed offline reference. Read this top-to-bottom on the flight to refresh the framework's design, where the project stands today, and which decisions are queued up. Keeps you productive without database / internet access.

Two companion files in this directory:

```
  CLAUDE.md                   — full project context, refreshed at session start
  RESOLVER_WALKTHROUGH.txt    — guided reading of resolve_scenario.py end-to-end
  HANDOVER.md                 — full session-by-session detail (verbose)
```

PART 1 — Core design principles (with the WHY for each)

These twelve principles are committed and must not be relaxed without explicit discussion. They are the foundation of the thesis AND the production deployment.

2.1 Asset-centric representation

Every physical asset is a node with its own inventory and mass balance. Pipelines are nodes (with line-fill). Vessels are nodes (with cargo). Refineries, terminals, storage hubs, gathering systems — all nodes.

WHY: The alternative (location-centric, with pipelines as edge attributes) hides in-transit crude as implicit state. In commodity logistics in-transit volumes are substantial, observable, and operationally meaningful — they cannot be hidden.

2.2 Zero-flow edge convention (stable topology)

Edges are permanent. Inactive edges carry flow=0, never absent. Same for nodes: inactive nodes stay in the asset graph, marked collapsed.

WHY: Stable adjacency matrix across time. Standard GNN architectures require it. Also: data upgrades never force a structural rewrite.

2.3 Universal mass balance with balancing item

Every node satisfies the SAME equation:

```
delta(S) = P + F_in - C - F_out + B
```

B is the balancing item — a first-class node variable that absorbs systematic reporting discrepancies. IEA convention.

WHY: One equation, one schema, no special cases. B retained because it is operationally informative (Hurricane Harvey 2017, COVID 2020, SPR releases 2022 all visible in B).

2.4 Formula-implies-edge

Edges never declared independently. They emerge from variable formulas. An edge (i, j, g) exists iff variable v(i, g) = f(..., v(j, g), ...).

WHY: The variables collection is the authoritative data model.

Edges, hierarchies, rollups are all derived views. One source of truth.

2.5 Persistent asset graph vs scenario graph
Two layers, cleanly separated:
- Persistent asset graph = fixed superset physical topology
- Scenario graph = derived at load time from asset graph + assignment table + coverage contract

WHY: Data upgrades NEVER force structural rewrite. Coverage contracts change scenarios; the asset graph is invariant. Operationally critical for production: bring new feeds online without breaking existing downstream consumers.

2.6 Physical layer vs observational aggregation layer
Physical nodes are real named assets with capacity and edges. Observational aggregates (PADDs, state-level rollups, basin views) are formula-defined views over physical nodes. They carry no edges in the physical graph.

WHY: Avoid double-counting. The PADD aggregate's mass balance is the sum of its children's, NOT a separate observation. Today's imports_agg cleanups put this principle in the data: TS authority belongs to the physical layer; the observational view derives.

2.7 Geography metadata != resolution hierarchy
Geography labels (lat/lon, county, state, PADD, country) are PROPERTIES. Multiple nodes share them. Rollups filter on them.

Resolution hierarchy is RELATIONSHIPS between nodes representing the same physical system at different granularities (Permian basin <-> Permian-TX <-> Midland-County <-> individual lease).

WHY: Conflating them creates double-counting (assigning data to basin AND state for same crude). Keeping them separate makes the observed/derived invariant enforceable at the schema level.

2.8 Observed/derived invariant — THE central guarantee
For each (node, variable, commodity, timestamp), exactly one of {observed_authoritative, derived} holds. Additional observed series may be attached as observed_auxiliary (stored on the node but not participating in mass balance at their own level).

Enforced at the SCHEMA level via UNIQUE constraint + enum status. Aggregation double-counting therefore cannot arise silently — it either throws at INSERT or is caught by the scenario loader's invariant check.

WHY: This is what the framework promises. Every other principle reinforces it. The TS-binding uniqueness audit (added 2026-05-13) extends this to the TS attribution level: one TS, one variable per scenario.

2.9 Coverage contracts
Per-scenario declarations of WHICH resolution level is authoritative for each subsystem. Tells the scenario loader: "for Permian, the basin aggregate is authoritative, sub-basin nodes are collapsed."

Also permits administrative aggregates (PADDs) to be PROMOTED to

authoritative when no finer data exists for some subsystem.

WHY: Production feature. Deployment can rescope which subsystems are observed at what level without rewriting the asset graph. Same framework, different contract = different scenario.

2.10 Collapsed state for junction nodes

Junction / gathering / pump-station nodes that physically exist but whose variables are indistinguishable from neighbours at current data resolution are collapsed. They REMAIN in the graph. Their flows pass through as formulas from authoritative upstream to authoritative downstream. When finer data lands, they wake up.

WHY: Forward-compatibility. No structural rewrite when granularity improves. The graph is the destination shape, populated to whatever resolution the current data supports.

2.11 Latent allocation at junctions

Where flows split across multiple routes and the per-route split is not observable in public data, the framework does NOT infer it through ad-hoc rules. It treats each per-edge flow as a latent quantity constrained by mass balance and capacity ceilings.

The downstream inference layer (LP solver, attention-based GNN, etc.) solves for these latents using mass balance + grade compatibility + capacity + slate observations as joint constraints.

WHY: The framework's job is to express constraints, not invent splits. This is what makes the schema usable for production trading: latent variables become DECISIONS for the inference layer to solve.

2.12 Bidirectional flows as separate edges

For reversible pipelines and bidirectional terminals (Seaway, Capline, LOOP, SPR sites), each direction is a SEPARATE directed edge with its own flow variable. Zero-flow convention handles the inactive direction.

WHY: One bidirectional edge would lose the directionality information that mass balance needs to close at the endpoints.

PART 2 — Architectural patterns that fall out of the principles

---

PATTERN A — The aggregate node pattern (Principle 2.6 in the data)

When a PADD-level EIA aggregate carries a flow that physically traverses multiple latent routes, create an abstract aggregate node that owns the TS:

```
source ──[TS]──> padd_X_agg ──> downstream (latent splits)
                      ^
                      |
padd_view.inflow <- source = alias(padd_X_agg.inflow <- source)
```

Examples already in the schema:

foreign_supply -> padd1_imports_agg	(TS: foreign_supply_to_padd1_kbd)
foreign_supply -> padd3_imports_agg	(TS: foreign_supply_to_padd3_kbd)

```
    foreign_supply -> padd5_imports_agg      (TS: foreign_supply_to_padd5_kbd)
    canadian_oil_sands -> padd2_canadian_imports_agg  (TS: MCRIPP2CA2)
                                              [NEW prototype, 2026-05-13]

  Benefits:
   - One TS per physical flow (Principle 2.8 at the TS level)
   - Physical-layer aggregate authoritative (Principle 2.6)
   - Natural future home for per-grade decomposition
   - The "gap" between observed parent and latent constituents IS the
     thesis Claim-4 signal — the magnitude of partition-internal latent
     flows the inference layer must solve.

PATTERN B — Pass-through nodes via node-type defaults

  Pipelines, terminals, gathering nodes all have node_type defaults:
    P = 0, C = 0, delta(S) = 0, B = 0
  Therefore: sum(inflows) = sum(outflows). The mass balance falls out
  of universal Principle 2.3 + the structural defaults.

  This is why a pipeline that delivers at A AND continues to B does NOT
  need to be split into two segments. One node, one inflow, multiple
  outflows, balance automatically holds.

  Segmentation is only correct when (a) per-segment data exists, (b)
  segments differ materially in capacity/operator/tariff, or (c) part of
  the route reverses while another part does not.

PATTERN C — Provenance loss at commingling, recovery via grade constraints

  Storage hubs (Cushing, Patoka, Houston, etc.) commingle crude — barrels
  become fungible in tankage. Provenance is LOCALLY destroyed. The
  framework correctly stops tracking molecular origin past these nodes.

  BUT: once the schema captures grade/quality at the source + along the way
  + per-refinery observed slate, provenance becomes GLOBALLY recoverable
  via joint constraints (mass balance + grade compatibility + capacity
  + slate). The framework provides the substrate; the downstream
  inference layer reconstructs.

  This is the production-deployment lens: the framework's "gaps" are
  unknowns the inference layer will solve, not framework deficiencies.

PATTERN D — The resolver (resolve_scenario.py)

  Topological walk over all variables in the scenario. For each variable,
  try eleven resolution rules in priority order; first match wins:

    1.  Observed (TS-bound)            -> source='observed'
    2.  Structural zero                -> source='zero'
    3.  Latent (+ mirror promotion)    -> source='latent' or 'derived' (mirror)
    4.  sum_over_children              -> source='derived'
    5.  sum_over_outflows              -> source='derived'
    6.  Single-variable alias          -> source='derived'
    7.  Reverse mirror (fallback)      -> source='derived'
    8.  Closure formula (B = ...)      -> source='derived'
    9.  Arithmetic (A - B - C)         -> source='derived'
   10.  Same-type rollup               -> source='derived'
   11.  Unresolved (target = 0)        -> source='unresolved'
```

See RESOLVER_WALKTHROUGH.txt PART 3B for the flow-of-resolution diagram.

PART 3 — Current state at a glance (2026-05-13 evening, run_id=11)

Schema:
    17 core tables + 8 views, 220 assets (196 physical + 24 abstract aggregates).
    409 directed flow edges. 1,700 variables. 137 EIA catalogue series.
    ~16,400 timeseries fact rows. 203,568 resolved rows in scenario_resolved_values.

Resolver dispatch (run_id=11):

observed	80	(perfect 1:1 with 80 distinct TS)
zero	544	
latent	637	
sum_over_children	4	
sum_over_outflows	1	
alias	411	
reverse_mirror	15	
closure	0	
arithmetic	8	
unresolved	0	

Audits clean:
    TS-binding uniqueness: 0 collisions (Principle 2.8 holds at TS level)
    Partition gap (DB-sourced tree): usa_view 0/0, padd1 229/92,
    padd2 1563/1234, padd3 2523/4264, padd4 280/932, padd5 0/0.
    Remaining gaps trace exactly to inter-PADD pipe corridors and
    Canadian-pipe splits where no aggregate exists yet — pure
    Principle-2.11 latent allocation.

What works end-to-end:
    EIA fetch -> oil_network.timeseries_data
    -> variable_assignments + node_type_default_formulas
    -> v_effective_assignments (override layer)
    -> resolve_scenario.py
    -> scenario_resolved_values + scenario_resolver_runs (audit log)
    -> HTML explorers (balance, hierarchy, map — *_resolver.html variants
       read scenario_resolved_values directly)

PART 4 — Next steps in priority order

A. Extend the aggregate pattern across remaining PADD 2 directions

   Builds on today's padd2_canadian_imports_agg prototype.
   New nodes to add:
     padd2_to_padd3_agg (TS: combined_inter_padd_P2_to_P3_kbd, 2020 kbd)
     padd3_to_padd2_agg (TS: combined_inter_padd_P3_to_P2_kbd, ~569 kbd)
     padd4_to_padd2_agg (TS: combined_inter_padd_P4_to_P2_kbd, ~979 kbd)

padd2_to_padd4_agg (TS: combined_inter_padd_P2_to_P4_kbd, ~280 kbd)
padd2_to_padd1_agg (TS: combined_inter_padd_P2_to_P1_kbd, ~19 kbd)
padd2_exports_agg (TS: MCREXP22, 107 kbd; no terminals yet)

Each closes another slice of the PADD 2 remaining gap.I = 1563 / gap.O = 1234. Same idempotent migration pattern as add_padd2_canadian_imports_agg.py.

B. Generalise to PADDs 1, 3, 4 inflow/outflow aggregates

Canadian aggregates where physical pipes exist:
padd1_canadian_imports_agg (146 kbd, no physical pipe in graph — placeholder)
padd3_canadian_imports_agg (431 kbd, Keystone south leg)

Inter-PADD aggregates for each cross-PADD corridor (mirror of section A). PADD 5 is mostly done — TMX migration handled it.

C. Append audit_ts_binding_uniqueness.py to the orchestrator

Add to initialize_oil_network_assignments.ipynb after resolve_scenario.py. Exit-1 on violations gates further automation. Five-minute change.

D. Grade / quality dimension (the next MAJOR axis)

Currently single-commodity (crude). Extend to:
variables.commodity -> (commodity, grade)
or add a separate "grade" column to variables, plus per-flow grade attribution.

This is the production-deployment unlock. Once grades land:
- Per-refinery slate observations can be bound
- Latent allocations become identifiable via grade compatibility
- Provenance recoverable even past commingling hubs
- The inference layer (LP / GNN attention) has the joint constraint system to solve over

Big change but mechanical, not architectural. The variables table already has commodity as a FK; extending to (commodity, grade) is a schema migration plus a regenerate of variable_assignments.

E. Chapter 5 Claim-4 view

v_balancing_item_check: per (scenario, node, date), compute the closure-derived B and the observed B side-by-side with a gap column. The gap IS the magnitude of partition-internal latent flows — the direct artefact for Claim 4. Annotate with events (Harvey 2017, COVID 2020, SPR 2022, Colonial 2021).

At 2024-12-01 today: USA closure-B ~= -2826, observed-B = -379; gap = 2447 kbd of latent intra-USA flow.

F. Retire inline-SQL HTML generators

Visual-compare *_resolver.html vs original. When identical, drop 100+ lines of inline CTE + JS arithmetic from make_balance_ui.py etc. Rename resolver variants to be the new canonical generators.

G. UI integration — show sum_out_implied in balance HTML

   v_node_balance_check.sum_out_implied gives the constraint-implied
   total at each junction. Surface it next to the latent individual
   outflows. One-line annotation per junction.

H. Add starter_basin scenario

   Trivially cheap now that node-type defaults exist — only override
   rows needed. Validates cross-scenario consistency (Chapter 5 Claim 1).

PART 5 — Open questions to think about on the flight

Q1. PADD 2 Canadian aggregate: should we route the latent pipes (Mainline-CA, Keystone) THROUGH it (changing pipe source from canada -> agg), or keep it as a parallel TS-holder (canada -> agg AND canada -> pipes both exist)?

    Today's prototype is the parallel form. The first form is cleaner but requires renaming all pipe inflow variables and updating downstream references. Decide before doing PADDs 1, 3 Canadian aggregates so the pattern is consistent.

Q2. Inter-PADD aggregates: same question. Combined_inter_padd_P2_to_P3_kbd is the EIA aggregate (pipeline + tanker). The constituents are Capline, MarketLink, Seaway-S, plus tanker movements. Should the aggregate node sit BETWEEN padd2_view and those pipes? Or alongside as a parallel TS-holder?

    For inter-PADD, the BETWEEN form means changing pipe SOURCES from individual PADD-2 hubs (Cushing, Patoka) to the aggregate node, which re-routes the physical topology. Pedro's earlier instinct (the aggregate-as-routing-point) leans toward this form.

Q3. PADD 1 Canadian (146 kbd) has no physical pipes modelled. Two options:
    (a) create padd1_canadian_imports_agg with no downstream pipes — a pure placeholder TS-holder, ready for future barge/rail/Portland-Montreal modelling
    (b) leave the TS on padd1_view directly until physical routes are added

    Option (a) is structurally consistent with the pattern; option (b) is less invasive.

Q4. Reverse_mirror cycle gate: paired_aliases_us covers one case. Are there other paired-direction relationships (e.g. paired aliases TO us via a transitive chain) that could still create cycles? Worth thinking through on the flight. CycleError would catch it but it'd be a nicer property if the build_deps logic was provably cycle-free.

Q5. Grade dimension: when added, every aggregate needs per-grade variables. Does the imports_agg outflow split become latent per-grade (canadian_oil_sands_HEAVY out of agg -> Whiting; canadian_oil_sands_LIGHT out of agg -> Bayway) or do we model grade-mixing inside the agg itself (agg has multi-grade inflow, single-grade outflows per route)?

    The former requires more variables but is closer to physical reality.
    The latter is simpler but loses information at the agg boundary.

Q6. Thesis vs production framing in Chapter 1 / 4: how explicit to be about
    the production-deployment lens? It is currently implicit (in CLAUDE.md
    and the memory file) but not in the thesis text. Worth a paragraph in
    the introduction setting the framing, or keep it cleanly academic?

 PART 6 — HTML files for offline browsing (all generated 2026-05-13)

In this directory, double-click any of these to open in a browser. All work
fully offline — they embed all data and JS inline. No internet needed.

  oil_network_balance_resolver.html   (917 KB)
    Per-node mass balance equation viewer.
    Pick any node, pick a date, see P/C/I/O/B/S/delta(S).
    Now includes padd2_canadian_imports_agg in the tree.

  oil_network_hierarchy_resolver.html   (605 KB)
    Drill-down tree explorer. Roots -> physical leaves.
    Shows resolved values from the resolver, not inline CTE.

  oil_network_map_resolver.html   (137 KB)
    Geographic projection. 196 physical assets + 376 flow edges.
    Click any edge midpoint for flow details.

  oil_network_balance.html, oil_network_hierarchy.html, oil_network_map.html
    Original (inline-SQL) versions. Kept for visual comparison until
    retired (next-steps item F).

  oil_network_explorer.html   (972 KB)
    Older general-purpose explorer.

 PART 7 — Quick references

PostgreSQL local connection:
    db=eia_crude, user=eia_user, password=eia_password, schema=oil_network

Run resolver:
    python resolve_scenario.py --scenario starter_us_crude_2015_2025

Run audits:
    python audit_ts_binding_uniqueness.py starter_us_crude_2015_2025
    python audit_partition_gaps.py

Add a new aggregate (template):
    See add_padd2_canadian_imports_agg.py — idempotent, ~120 lines.

Regenerate HTMLs:
    python make_balance_resolver_ui.py
    python make_hierarchy_resolver_ui.py
    python make_map_resolver_ui.py

END

Good flight. When you're back, the queued next-step that gives the biggest visible payoff is (A) — finishing the PADD 2 aggregate set. The PADD 2 gap.I will drop from 1,563 -> close to 0 once those aggregates exist. That makes a clean "before vs after" narrative for Chapter 5.